

NATIONAL ECONOMICS UNIVERSITY  
FACULTY OF DATA SCIENCE & AI  
COLLEGE OF TECHNOLOGY

OFFICIAL ASSIGNMENT PAPER  
Academic Year: 2026–2027 Semester: Fall 2026  
Course Code: DSAI1002

## DATA STRUCTURES AND ALGORITHMS WITH PYTHON

Topic: Sorting, Selection, and Searching | Paper Code: 102 | Total Marks: 10.0  
Points

Student Full Name:

Student ID (MSSV):

Class / Cohort: ....

Submission Date: ...

| Question  | Q1  | Q2  | Q3  | Q4  | Q5  | Q6  | Q7  | Q8  | Total (10.0) | Examiner Signature |
|-----------|-----|-----|-----|-----|-----|-----|-----|-----|--------------|--------------------|
| Max Score | 1.0 | 1.0 | 1.0 | 1.5 | 1.5 | 1.0 | 1.5 | 1.5 | 10.0         |                    |
| Score     |     |     |     |     |     |     |     |     |              |                    |

**Instructions:** Write responses clearly and concisely within the designated answer boxes. Show intermediate steps and traces where requested. Do not detach pages.

### Part I: Conceptual Foundations & Theoretical Principles (3.0 Points)

#### Question 1 (1.0 Point) – Sorting Classification, Stability & Memory Overhead

- (a) (0.5 pts) Define what it means for a sorting algorithm to be **stable**. State one practical scenario where stability is required. Give a 3-element counterexample showing why **Selection Sort** is inherently unstable.
- (b) (0.5 pts) Define what an **in-place** sorting algorithm is. Classify the auxiliary memory requirements (Auxiliary Space) of **Merge Sort**, standard **Quicksort**, and **Insertion Sort**.

#### Candidate Response – Question 1

#### Question 2 (1.0 Point) – Information-Theoretic Lower Bound for Comparison Sorts

- (a) (0.5 pts) Explain the **decision tree model** for comparison sort on  $n$  distinct elements. Why must any valid comparison sort tree contain at least  $n!$  leaves? Deduce the minimum tree height  $h$ .
- (b) (0.5 pts) Formally derive the  $\Omega(n \log n)$  lower bound using  $\sum_{i=1}^n \log_2 i$ . How do non-comparison algorithms (e.g., Counting Sort) bypass this theoretical lower bound?

#### Candidate Response – Question 2

**Question 3 (1.0 Point) – Binary Search Invariants & Integer Overflow**

- (a) **(0.5 pts)** Explain why the textbook midpoint formula `mid = (low + high) // 2` can cause a critical runtime failure (integer overflow) in fixed-width arithmetic languages (e.g., Java/C++) when searching large arrays ( $n \approx 1.5 \times 10^9$ ). Write the overflow-safe expression and explain why it guarantees correctness.
- (b) **(0.5 pts)** Describe how standard Binary Search must be modified to locate the **first occurrence** (lower bound) of a target key in an array with duplicate elements, while strictly maintaining  $O(\log n)$  runtime.

**Candidate Response – Question 3****Part II: Algorithmic Traces & Execution Analysis (4.0 Points)**

**Question 4 (1.5 Points) – Counting Sort Mechanics & Stability Preservation** Given the input array  $A = [4, 2_a, 2_b, 8, 3_a, 3_b, 1]$  of length  $n = 7$  with values in range  $k \in [1..8]$ .

- (a) **(0.5 pts)** Construct the frequency histogram array  $C$  (size 9 for indices 0..8) and the cumulative prefix sum array  $P$  ( $P[v] = \sum_{j=0}^v C[j]$ ).
- (b) **(0.5 pts)** Trace step-by-step the population of the output array  $B$  as  $A$  is processed **from right to left** ( $j = 6$  down to 0). Show the decrements to  $P$  and the final content of  $B$ .
- (c) **(0.5 pts)** Mathematically prove why scanning from *right to left* guarantees stability, and state what would happen to duplicate elements if  $A$  were scanned from *left to right* with decrement.

**Candidate Response – Question 4**

**Question 5 (1.5 Points) – Lomuto Partitioning & Quicksort Recurrence Trees** Given the array  $A = [5, 2, 9, 3, 7, 6, 4]$  ( $low = 0, high = 6$ ), with pivot  $p = A[6] = 4$ .

- (a) **(0.75 pts)** Trace the exact step-by-step execution of **Lomuto's Partitioning Algorithm**. For each step of loop index  $j \in \{0, 1, \dots, 5\}$ , specify pointer  $i$ , whether a swap occurs, the resulting array state, and the final pivot placement swap.
- (b) **(0.75 pts)** State the recurrence relation and recursion tree depth for Quicksort in: (1) the **best case** (balanced splits), and (2) the **worst case** (already sorted with extreme pivot). Derive the asymptotic complexity for each case.

Candidate Response – Question 5

**Question 6 (1.0 Point) – Quickselect vs. Heap for Top- $k$  Extraction** Consider finding the top  $k = 50$  largest values from a dataset of size  $n = 10,000,000$ .

- (a) **(0.5 pts)** Explain the pruning mechanism of **Quickselect** and why its expected runtime is  $\Theta(n)$ , whereas Quicksort requires  $\Theta(n \log n)$ .
- (b) **(0.5 pts)** Compare the **Min-Heap of size  $k$**  approach against **Quickselect** in terms of Time Complexity, Auxiliary Space, and operational fitness when processing an incoming live data stream.

Candidate Response – Question 6

**Part III: Algorithmic Design & Problem Solving (3.0 Points)**

**Question 7 (1.5 Points) – Two-Sum on a Sorted Array** Given a 1-indexed array of integers  $A$  that is already sorted in non-decreasing order ( $A[1] \leq A[2] \leq \dots \leq A[n]$ ) and a target sum  $S$ , design an algorithm to find two distinct indices  $i < j$  such that  $A[i] + A[j] = S$ .

- (a) **(0.6 pts)** Write the algorithm in Python using the **Two-Pointer Technique**.
- (b) **(0.5 pts)** State the loop invariant and prove why advancing the left pointer or retreating the right pointer is guaranteed never to discard a valid solution pair.
- (c) **(0.4 pts)** Analyze the worst-case time complexity and auxiliary space complexity. Why is this superior to using a Hash Map on sorted inputs?

**Candidate Response – Question 7**

**Question 8 (1.5 Points) – Searching in a Rotated Sorted Array** An array of distinct integers sorted in ascending order is rotated at an unknown pivot index (e.g.,  $[4, 5, 6, 7, 0, 1, 2]$ ). Given a target value, find its index in  $O(\log n)$  time.

- (a) **(0.7 pts)** Implement the modified binary search algorithm in Python. Clearly explain how you identify which half is sorted.
- (b) **(0.5 pts)** Trace your algorithm on  $A = [6, 7, 1, 2, 3, 4, 5]$  with target = 1. Show values of *low*, *mid*, *high* at each iteration.
- (c) **(0.3 pts)** Formally justify why at least one half is always monotonically sorted, and state the recurrence relation that proves  $O(\log n)$  time.

**Candidate Response – Question 8**